माधव प्रौद्योगिकी एवं विज्ञान संस्थान, ग्वालियर (म.प्र.), भारत
MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR (M.P.), INDIA
Deemed University
(Declared under Distinct Category by Ministry of Education, Government of India)
NAAC ACCREDITED WITH A++ GRADE

WEB TECHNOLOGY

30261104

COURSE OBJECTIVES

To introduce students to the fundamentals of web development, including HTML, CSS, JavaScript, and XML for building structured web pages.

To develop the ability to design responsive and interactive web interfaces using modern tools and frameworks.

To familiarize students with basic web hosting, deployment, and security concepts for realworld web applications.

Unit I: Introduction to Web Development & HTML: Overview of Web, Technologies (HTML, CSS, JavaScript), Structure of an HTML Document, Basic HTML Tags (Headings, Paragraphs, Lists, Links, Images), Tables and Forms in HTML

CO1. design and structure web pages using HTML and XML concepts

by: Dr. Shradha Dubey
Assistant Professor
Department of CCST

Web

- **Definition:** The World Wide Web (WWW) is a system of interconnected documents and resources linked by hyperlinks and accessed via the Internet.
- The World Wide Web (WWW) is a collection of information, resources, and services that are interconnected using hyperlinks and accessed via the Internet. It is made up of billions of web pages stored on computers (servers) worldwide and viewed using a web browser.
- **How it works:**
  - Websites are created using HTML, CSS, and JavaScript.
  - These websites are stored on servers.
  - Users access them through a web browser (Chrome, Firefox, Edge) using HTTP/HTTPS.

- Invented by **Tim Berners-Lee** in **1989** at CERN.
- Works on the **Internet**, but it is **not the same as the Internet**.
- Uses **HTTP/HTTPS protocols** to transfer data.
- Documents are written in **HTML** and linked using **hyperlinks**.
- Accessed using **web browsers** like Chrome, Firefox, Edge, or Safari.

- **Example:** When you type *www.google.com* in your browser, you are accessing a web resource from Google's servers.
- When you search for something on Google, click a link on Wikipedia, or shop on Amazon, you are browsing the **World Wide Web**.

Technology

- **Definition:** Technology is the application of scientific knowledge, tools, and techniques to solve problems or perform tasks efficiently.
- **Scope:** It includes machines, software, communication systems, and processes that make our lives easier.
- **Example:**
  - A mobile phone is technology that helps in communication.
  - Databases are technologies that help in storing and managing information.

Web Technology (Combined Term)

- **Tools and techniques used to create websites that run on the internet**
- **Definition:** Web Technology refers to the set of tools, techniques, and protocols used for creating, delivering, and managing web-based content and applications.
- **Examples:**
    - HTML, CSS, JavaScript (building web pages)
    - HTTP/HTTPS (communication protocol)
    - Web servers (Apache, Nginx)
    - Databases (MySQL, MongoDB)
- **In simple words:** Web Technology is everything that makes websites and web applications work, look good, and interact with users.
- Mechanism by which computer can communicate with each other.
- It provides higher and faster mutual device communication
- **Tools and techniques used to create websites that run on the internet**

Internet	**World Wide Web (WWW)**
A global network of interconnected computers and devices.	A collection of information (websites/pages) accessed via the Internet.
Provides infrastructure (hardware, cables, satellites).	Provides content (documents, media, web apps).
Exists since the 1960s.	Introduced in 1989.
Includes services like email, file transfer, VoIP, IoT, etc.	Only the websites and web applications.

It includes:

- Web Browser
- HTML
- CSS
- JavaScript
- Webserver

Web Browser

- **Definition:** A web browser is a **software application** used to access, retrieve, and display web pages from the World Wide Web.
- **Examples:** Google Chrome, Mozilla Firefox, Microsoft Edge, Safari.
- **Function:**
    - Sends a request to the web server using **HTTP/HTTPS**.
    - Receives the response (HTML, CSS, JavaScript, images).
    - Renders it as a web page the user can see and interact with.

HTML (HyperText Markup Language)

- **Definition:** The **standard markup language** used to create the **structure** of web pages.
- **Role:** Defines headings, paragraphs, images, links, tables, etc.
- **Example:**
- <h1>Welcome</h1>
- This is a paragraph.
- Click Here

CSS (Cascading Style Sheets)

- **Definition:** A style sheet language used to **design and format** the appearance of web pages.
- **Role:** Controls colors, fonts, layouts, spacing, responsiveness.

- **Example:**

```
body {
  background-color: lightblue;
  font-family: Arial, sans-serif;
}
h1 {
  color: darkblue;
  text-align: center;
}
```

JavaScript

- **Definition:** A **programming language** that makes web pages **interactive and dynamic**.
- **Role:** Adds logic, behaviors, and functionality to web pages.
- **Example:**
- alert("Hello! Welcome to the Website");

Web Server

- **Definition:** A **computer system/software** that stores websites and delivers them to users over the Internet.
- **Examples:** Apache, Nginx, Microsoft IIS.
- **Function:**
    - Waits for requests from browsers (clients).
    - Sends back web pages or web applications.
- **Analogy:**
  Think of a **restaurant**:
    - You (browser) order food.
    - The **kitchen (web server)** prepares it.
    - The waiter delivers it back to you.

Web technology Strategies

- Approaches or plans used to create effective and user-friendly website

Strategies

- **Frontend (client Side)**
- **Backend (Server Side)**

Client-Side (Frontend)

The part of a web application that runs in the user's browser.

- **Definition:** Client-side refers to everything the user sees and interacts with directly on the screen.
- **Technologies Used:**
  - HTML → Structure (content)
  - CSS → Presentation (style, design, colors, fonts, layout)
  - JavaScript → Interactivity (buttons, forms, animations, dynamic updates)
- **Role:**
  - Display information received from the server.
  - Handle user input (clicks, typing, forms).
  - Perform small computations (e.g., validating a form before sending to server).
- **Examples:**
  - When you click a button on Amazon, the button highlight effect (CSS + JS) is client-side.
  - A popup message saying "Please enter your email" before submitting a form is client-side validation.

The part of a web application that runs on the web server, hidden from users.

- **Definition**: Server-side refers to all the operations that happen behind the scenes to process requests and deliver responses.
- **Technologies Used:**
  - Languages: PHP, Python, Java, Node.js, Ruby, C#.
  - Frameworks: Django, Spring Boot, Express.js, Laravel.
  - Databases: MySQL, PostgreSQL, MongoDB.

- **Role:**
  - Process user requests (e.g., logging in, saving form data).
  - Communicate with databases (store/retrieve information).
  - Apply business logic (e.g., calculate bill, recommend products).
  - Send back processed data (usually in HTML, JSON, or XML format).
- **Examples:**
  - When you log in to Gmail, the server checks your username & password in the database.
  - When you search on Flipkart, the server retrieves the relevant products and sends them to your browser.

How They Work Together (Flow)

1. You (client/browser) send a **request** → e.g., type *www.amazon.in*
2. The **server** receives the request and processes it.
3. The **server** fetches data from the database.
4. The **server** sends back a **response** (HTML + CSS + JS + data).
5. The **client (browser)** renders it so you can see and interact.

Feature	Client-Side (Frontend)	Server-Side (Backend)
Runs On	User's browser	Web server
Technologies	HTML, CSS, JavaScript, React, Angular, Vue	PHP, Python, Java, Node.js, Ruby, Databases
Visible to User	Yes (users see UI)	No (hidden from users)
Main Role	Design, interaction, user experience	Logic, database handling, authentication
Examples	Form validation, animations, dropdown menus	Login system, shopping cart, payment process

History of the World Wide Web

1. Pre-Web Era (1960s – 1980s)

- **1960s:** Internet's foundation was laid with **ARPANET** (Advanced Research Projects Agency Network) in the USA.
- **1970s–80s:** Protocols like **TCP/IP** were developed, enabling global computer communication.
- Internet existed, but **no websites, no browsers, no hyperlinks**.

2. Birth of the Web (1989 – 1991)

- **1989: Tim Berners-Lee**, a British scientist at CERN, proposed the idea of the **World Wide Web**.
- He introduced:
  - **HTML (HyperText Markup Language)** → for structuring documents.
  - **HTTP (HyperText Transfer Protocol)** → for communication.
  - **URL (Uniform Resource Locator)** → for addressing resources.
- **1991:** First website (info.cern.ch) went live.

3. Early Growth (1993 – 1999)

- **1993:** First popular browser **Mosaic** was released (later became Netscape Navigator).
- **1994:** World Wide Web Consortium (**W3C**) was founded by Tim Berners-Lee to standardize web technologies.
- **Mid-1990s:**
  - Rise of **Netscape Navigator** (dominant browser).
  - Microsoft introduced **Internet Explorer**.
  - First search engines (Yahoo!, AltaVista, Lycos).
- Websites were simple: **text + images + hyperlinks**.

4. Web 1.0 (Static Web) – 1990s

- Read-only web.
- Static HTML pages.

- Limited interaction (no login, no social media).

5. Web 2.0 (Interactive Web) – 2000s

- Web became **dynamic & interactive**.
- Technologies: **JavaScript, AJAX, PHP, MySQL**.
- Rise of **social media, blogs, e-commerce**.
- Examples: Facebook, YouTube, Wikipedia, Amazon, Google.

6. Web 3.0 (Semantic & Intelligent Web) – 2010s onwards

- More personalized and intelligent web.
- Use of **AI, machine learning, big data**.
- Mobile-first design, responsive layouts.
- Cloud computing & APIs.
- Examples: Chatbots, Recommendation systems, Smart assistants (Siri, Alexa).

7. Web 4.0 (Future Web / Intelligent & Ubiquitous Web)

- Still evolving.
- Aims for **fully intelligent, interconnected systems**.
- Integration of **IoT, AR/VR, Blockchain, Metaverse**.
- Humans + machines will interact seamlessly.

Year/Period	Key Event
1960s	ARPANET (early Internet)
1989	Tim Berners-Lee proposes the Web
1991	First website goes live
1993	Mosaic browser released
1994	W3C founded
1995–2000	Search engines, e-commerce, dynamic pages

Year/Period	Key Event
2000s	Web 2.0 (social media, interactivity)
2010s	Web 3.0 (AI, mobile, cloud)
Future	Web 4.0 (IoT, AR/VR, metaverse)

- **Web 1.0 = Reading (static pages).**
- **Web 2.0 = Writing & Sharing (social, interactive).**
- **Web 3.0 = Intelligent (AI-driven, personalized).**
- **Web 4.0 = Future (everywhere, immersive, connected world).**

Protocols Governing the Web

A **protocol** is a set of rules that allows computers to communicate with each other. On the **World Wide Web**, multiple protocols work together to make sure we can **browse, search, and transfer information smoothly**.

Key Protocols

HTTP (HyperText Transfer Protocol)

- The **main protocol of the Web**.
- Defines how web browsers and servers **communicate**.
- Used for transferring web pages (HTML, CSS, JS, images).
- Works on **port 80**.
- Example: http://www.example.com

HTTPS (HyperText Transfer Protocol Secure)

- Secure version of HTTP.
- Uses **SSL/TLS encryption** to protect data (like passwords, banking info).
- Works on **port 443**.
- Example: https://www.bank.com
- Nowadays, most websites use **HTTPS** for security.

TCP/IP (Transmission Control Protocol / Internet Protocol)

- The **foundation of Internet communication**.
- **TCP:** Breaks data into small packets and ensures they are delivered correctly.
- **IP:** Ensures packets go to the right destination (via IP addresses).
- Without TCP/IP, no communication between browser and server is possible.

DNS (Domain Name System)

- Like the **phonebook of the Internet**.
- Converts human-readable names (like *www.google.com*) into machine-readable **IP addresses** (like 142.250.183.206).
- Without DNS, we would have to remember numbers instead of names.

FTP (File Transfer Protocol)

- Used for **uploading and downloading files** between client and server.
- Example: A web developer uses FTP to upload a website's files to the hosting server.

SMTP/IMAP/POP3 (For Emails – Web-related services)

- **SMTP (Simple Mail Transfer Protocol):** For sending emails.
- **IMAP (Internet Message Access Protocol):** allows users to access and view email messages stored on remote servers
- **FTP → File sharing**
- **SMTP/IMAP → Emails**

Writing a Web Project

Define project idea

Plan

Choose tools

Start coding

Test your website

Deploy your website

Get feedback

So in short:
Idea → Plan → Tools → Coding → Testing → Deployment → Feedback → Improvement

1. Define Project Idea

- First, you decide **what your website will be about**.
- Example: An online food recipe blog, an e-commerce site, or a personal portfolio.
- Clearly mention **purpose, audience, and features**.
  This step gives direction to your project.

2. Plan

- Break your idea into **smaller tasks** (e.g., homepage, contact page, login system).
- Create a **rough design** (wireframe/sketch).
- Plan the **timeline**: which part will be done first, which later.
  Proper planning avoids confusion later.

3. Choose Tools

- Select the **technologies** you will use.
  - **Frontend**: HTML, CSS, JavaScript (React, Angular, or Vue if advanced).
  - **Backend**: Node.js, Django, Flask, PHP, etc.
  - **Database**: MySQL, MongoDB, PostgreSQL.
  - **Editor**: VS Code, Sublime.
    Tools depend on your project's complexity and your skills.

4. Start Coding

- Begin by setting up the **basic structure**.
- Build your pages and add functionality step by step.
- Keep your code **organized and modular**.
  This is where your plan starts turning into reality.

5. Test Your Website

- Check if everything works properly.

- Test on different **browsers (Chrome, Firefox, Edge)**.
- Test on different **devices (mobile, tablet, desktop)**.
- Fix **bugs, errors, or design issues**.
  Testing ensures good **user experience**.

6. Deploy Your Website

- Put your website **online** so others can access it.
- Use hosting services like **GitHub Pages, Netlify, Vercel, or AWS**.
- Make sure your domain name is easy to remember.
  Deployment makes your project live for users.

7. Get Feedback

- Ask users, friends, or mentors to **try your website**.
- Collect their **suggestions and issues**.
- Improve design, performance, and features based on feedback.
- Feedback helps in making your project **better and more user-friendly**.

Idea → Plan → Tools → Coding → Testing → Deployment → Feedback → Improvement

HTML INTRODUCTION

- HTML stands for Hyper Text Markup Language
- HTML is not a case sensitive.
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

H → Hyper

- "Hyper" means **beyond** or **more than normal**.
- **Reference/ link**
- In HTML, it refers to **hypertext**, which is text containing **links** that connect to other texts, documents, or resources.
- Example: When you click a blue underlined link on a webpage, that's **hypertext**.

T → Text

- Refers to the **content (words, sentences, paragraphs)** we write.
- But in HTML, text can also include **multimedia (images, videos, audio)** alongside written content.

Hypertext (Hyper + Text)

- **Text** → written content (words, sentences, paragraphs).
- **Hyper** → means *beyond* or *extended*.

So, **hypertext = text that goes beyond normal text** → it contains **links (called hyperlinks)** that connect to other texts, documents, or resources.

Examples

(i) In a Wikipedia article, when you click on a blue underlined word, it takes you to another related page → that's **hypertext**.

In HTML: Click here

(ii) A **normal book** → you read page by page in order.

A **hypertext document (like a website)** → it's like a *choose your own adventure* book where you can jump to any section instantly by clicking a link.

→ This is a **hyperlink**, a core part of hypertext.

M → Markup

- "Markup" means **special tags** that are added to text to tell the browser **how to display it**.
- Example:
  - <p> marks a **paragraph**.
  - <h1> marks a **heading**.
- Markup ≠ programming. It is about **structuring and formatting content**.

L → Language

- A **set of rules and syntax** that web browsers understand.
- Not a programming language (like Python or C), but a **markup language** used to design the structure of a webpage.

Does HTML have a compiler or interpreter?

The answer is **NO**.

- **HTML is not a programming language**; it is a **markup language**.
- It does not need a compiler or an interpreter like C (compiler) or Python (interpreter).

Then how does HTML work?

- A **web browser** (like Chrome, Firefox, Edge) acts as a **renderer** (a component (software, hardware, or person) that transforms information into a specific, often visual, form**)**.
- The browser **reads (parses) HTML code** line by line and **displays the structured content** (text, images, links, etc.) on the screen.
- So, instead of "compiling" or "interpreting," HTML is **rendered** by the browser.

Key difference

Language	Processed by	Example
C, C++	Compiler	Needs compilation before running
Python, JavaScript	Interpreter	Runs line by line
HTML	Browser (Renderer/Parser)	Displayed as web page

"HTML does not have a compiler or interpreter. It is a markup language, and the web browser itself parses and renders HTML to display web pages."

Compiler And an Interpreter

Both a **compiler** and an **interpreter** are programs that help a computer understand and run code written in programming languages (like C, Java, Python).

Compiler

- Translates the **entire program** (source code) into **machine code** (executable) **before running** it.
- The result is a separate file (e.g., .exe on Windows) that you can run later.
- The compiler (like GCC) translates this whole code into machine code first.
- If there are errors, you get them all together after compilation.
- Runs **faster** once compiled.

Interpreter

- Translates and runs the code **line by line** (one statement at a time).
- No separate executable file is produced — it runs directly.

Example: **Python, JavaScript**

- The Python interpreter reads the first line, executes it, then moves to the next.
- Errors are shown immediately at that line.
- Runs **slower** compared to compiled languages (since translation happens during execution).

Feature	Compiler	Interpreter
Execution	Translates whole program at once	Translates line by line
Speed	Faster after compilation	Slower (interprets while running)
Errors	Shows all errors together after compilation	Shows error immediately at the line

Feature	Compiler	Interpreter
Output	Creates a separate executable file	No executable file, runs directly
Examples	C, C++	Python, JavaScript

Compiler = like translating an entire book into Hindi before reading it.

Interpreter = like a live translator translating each line as you read.

HTML consists of:

HTML TAG

In HTML, Tags are the starting and ending parts of an HTML element. They begin with < symbol and end with > symbol. Whatever is written inside < and > are called tags.

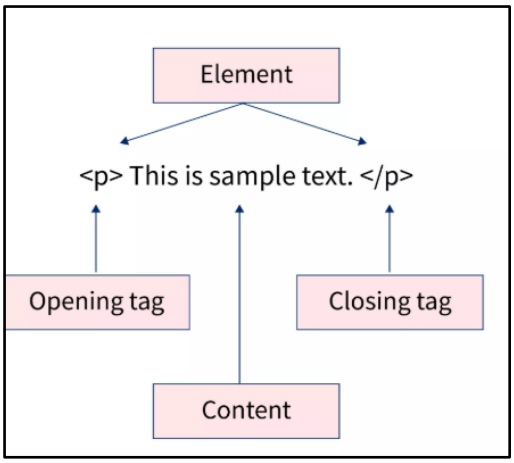
Example: In this example is the starting tag and is the ending tag.

HTML elements

The HTML element consists of both the opening and closing tags as well as what is *inside* those tags. It normally consists of some structure that is used to define the respective tags.

Example: In this example is the starting tag and is the ending tag but with content inside it.

This is the content.



HTML Attributes

HTML Attributes is used to define the character of an HTML element. It always placed in the opening tag of an element. It generally provides additional styling (attribute) to the element.

Example: In this example ** ** is the starting tag and ** ** is the ending tag with extra css attributes.

This is paragraph.

HTML Element and Tag

- <head> → **Opening tag** (tells browser: start the head section).
- </head> → **Closing tag** (tells browser: end the head section).
- The / (forward slash) shows that this is the **end** of the element, not a new one.

HEAD TAG

The <head> element is a container for metadata (data about data) and is placed between the <html> tag and the <body> tag.

Metadata is data about the HTML document. Metadata is not displayed.

Metadata typically define the document title, character set, styles, scripts, and other meta information.

The following elements can go inside the <head> element:

- <title> (required in every HTML document)
- <style>
- <base>
- <link>
- <meta>
- <script>
- <noscript>

Paragraph Tag

A paragraph in HTML is simply a block of text enclosed within the <p> tag.

The <p> tag helps divide content into manageable, readable sections. It is the go-to element for wrapping text in a web page that is meant to be displayed as a distinct paragraph.

Properties of the paragraph Tag

- The browser reduces multiple spaces added by users to a single space.
- If a user adds various lines, the browser compresses them into one line.
- By default, the display of the paragraph element is set to "block," meaning each new paragraph is placed on a new line.
- It is not an empty tag.

Code Example

```
<html>
<head>
    <title>HTML and Protocol Strategies</title>
</head>
<body>
  <h1>HTML and Protocol Strategies</h1>

  <p>
    HTML stands for HyperText Markup Language.
    It is the standard language used to create web pages.
    HTML structures the content using tags and attributes.
    It forms the backbone of every website on the internet.
  </p>

  <p>
    Protocols define the rules for communication on a network.
```

The most common protocol on the web is HTTP (Hypertext Transfer Protocol).

Protocols ensure that data is transmitted securely and reliably.

They make it possible for browsers and servers to interact smoothly.

HTTPS is a secure version of HTTP.

This prevents unauthorized access and data theft.

HTTPS is essential for e-commerce and online banking websites.

The frontend is the part of a website users see and interact with.

It includes design, layout, and user experience elements.

The backend handles server-side logic, databases, and security.

Together, frontend and backend make a complete web application.

</body>

</html>

Empty Tag in HTML (void element)

- In **HTML**, an **empty tag** (also called a **void element**) is a tag that **does not have any content** between an opening and closing tag.
- It is written **without a closing tag**.

Examples of empty tags in HTML

1.
 → line break
2. <hr> → horizontal rule (line)
3. <img> → image
4. <input> → form input field
5. <meta> → metadata
6. <link> → link to external resource (like CSS)
7. <source> → for audio/video sources
8. <area> → clickable area in an image map

Example Code:

```
<html>
<head>
  <title>Empty Tag Example</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
  <br> <!-- Empty tag: adds a line break -->
  <img src="flower.jpg" alt="Flower"> <!-- Empty tag: shows an image -->
  <hr> <!-- Empty tag: draws a horizontal line -->
</body>
</html>
```

Key Points:

- Empty tags **don't need closing tags** like </br> or </img> (in HTML5).
- In **XHTML**, you may see them written with a slash:
, <img />.
- They are used for **standalone elements** (like breaks, images, metadata) that don't wrap text/content.

So, **empty tag = tag with no closing part & no content inside**.

Empty Tag	**Purpose / Use**
 	Inserts a line break (new line).
<hr>	Inserts a horizontal line (thematic break).
<img>	Displays an image.
<input>	Creates an input field in forms (text, password, checkbox, etc.).
<link>	Links external resources like CSS files.
<meta>	Defines metadata (charset, description, keywords, viewport, etc.).
<source>	Defines media sources for <audio> or <video>.
<track>	Adds captions/subtitles to <video> or <audio>.
<area>	Defines clickable areas inside an image map.
<base>	Sets a base URL for all relative links in the document.
<col>	Defines column properties inside <colgroup> in tables.
<embed>	Embeds external content (e.g., multimedia, Flash, PDFs).
<wbr>	Suggests a line break opportunity (word break).

The
 tag

- The HTML
 tag element creates a line break, giving you a new line without starting a new paragraph.
- The paragraph tag treats multiple lines into a single line that's why we use br tag for this purpose.
- Use
 when you want to move to the next line without beginning a whole new paragraph.

```
<p>

  This paragraph has multiple

  <br >lines. But HTML reduces them

  <br >to a single line, omitting

  <br >the carriage return we have used.

</p>
```

The
 tag

• The HTML
 tag element creates a line break, giving you a new line without starting a new paragraph.

• The paragraph tag treats multiple lines into a single line that's why we use br tag for this purpose.

• Use
 when you want to move to the next line without beginning a whole new paragraph.

```
<p>
This paragraph has multiple
<br >lines. But HTML reduces them
<br >to a single line, omitting
<br >the carriage return we have used.
</p>
```

Comments in HTML

In **HTML**, a **comment** is a piece of text that is ignored by the browser (not shown on the webpage).
It is only for developers to add notes, explanations, or temporarily disable code.

Syntax of comment in HTML:

<!-- This is a comment -->

Examples:

1. **Single-line comment**

<!-- This is a heading -->

<h1>Hello World</h1>

2. **Multi-line comment**

<!--

This section contains different heading tags.

They go from h1 (largest) to h6 (smallest).

-->

<h1>Heading 1</h1>

<h2>Heading 2</h2>

3. **Commenting out code (to disable it temporarily)**

<!-- <p>This paragraph will not be displayed</p> -->

Code Example:

```
<html>
<head>
    <title>HTML Comment Example</title>
</head>
<body>
  <!-- This is a heading -->
  <h1>Welcome to My Website</h1>

  <!-- This is a paragraph -->

 <p>This is an example of a paragraph in HTML.</p>
  <!-- The image below will be displayed -->
  <img src="https://www.image.com/" alt="Example Image">

  <!--
    This is a multi-line comment.
    You can write notes here for better understanding.
  -->
</body>
</html>
```

1. Write an HTML program to display **"Welcome to HTML"** as a heading and add a comment explaining the <h1> tag.
2. Write an HTML program that displays your **name, course, and college name**, and use comments to identify each section.
3. Write an HTML program containing **three headings and three paragraphs**. Add an HTML comment before each heading.
4. Write an HTML program to create a simple **student introduction page** and use comments to explain the <head> and <body> sections.
5. Write an HTML program to display:
6. Name
7. Enrollment Number
8. Branch
9. Semester
10. Add appropriate comments before each element.
11. Write an HTML program with a heading and paragraph, but **comment out the paragraph** so that it does not appear on the webpage.
12. Write an HTML program containing **single-line and multi-line comments** and observe which content is displayed in the browser.
13. Write an HTML program for a simple **college webpage** with a title, heading, and paragraph. Use comments to describe each part of the code.
14. Write an HTML program with **two paragraphs**, but use a comment to temporarily hide the second paragraph.
15. Write an HTML program to create a webpage with the heading **"My First Web Page"**, a paragraph, and an image. Add comments explaining each HTML element.

Formatting Tags in HTML

Formatting tags are used to change the **appearance of text** (bold, italic, underline, etc.).

DIFFERENCE BETWEEN PRESENTATIONAL AND SEMANTIC TAGS

1. Presentational Tags

- These tags **only control the appearance** (look/style) of the text.
- They **do not carry meaning** about the content.
- Mostly used in **old HTML (before HTML5)**.
- Example:
- Bold Text
- Italic Text
- <u>Underlined Text</u>

Problem: Screen readers, search engines, and assistive technologies do not understand *why* the text is styled, only *how* it looks.

2. Semantic Tags

- These tags **convey meaning + style**.
- They tell the browser, developer, and screen readers **why** text is marked.
- Introduced in **HTML5** for accessibility and SEO.

Example:

- Important Text <!-- Strong importance -->
- Emphasized Text <!-- Stress/emphasis -->
- <mark>Highlighted Text</mark> <!-- Marked/highlighted -->
- ~~Deleted Text~~ <!-- Removed content -->
- <ins>Inserted Text</ins> <!-- Added content -->

Benefit: A screen reader will **read them differently**.

- → read with strong emphasis (like a louder voice).
- → read with stress (like italics in speech).

Comparison Table

Aspect	**Presentational Tags**	**Semantic Tags**
Purpose	Only changes the *appearance*	Adds *meaning* + *appearance*
Accessibility	Not accessible to screen readers	Accessible, improves readability
SEO Impact	No impact on search engines	Helps search engines understand content
Examples	, <i>, <u>, <font>,	, , <mark>, <del>, <ins>
HTML5 Recommendation	Deprecated / not recommended	Recommended

- Use **semantic tags** whenever possible → they make your HTML **meaningful, accessible, and SEO-friendly**.
- Use **presentational tags** only if absolutely needed (but CSS is better for styling).

Here are the **main formatting tags** in HTML

1. Bold Text

This is bold

This is strong (semantic bold)

- → just makes text bold (visual only).
- → makes text bold and adds importance (used by screen readers).

Frontend (Browser Display)

By default:

- → renders text **bold**.
- → also renders text **bold**.

So visually (frontend), both look the same in most browsers.

But the real difference is:

1. **Semantics**
   - <b> → no extra meaning, just style.
   - → carries semantic meaning: *importance*.
2. **Accessibility (Screen Readers)**
   - <b> → read normally.
   - → read with stress/emphasis (e.g., louder/stronger intonation).
3. **SEO**
   - Search engines consider as giving weight/importance to words.
   - <b> is treated as just styling with little to no semantic weight.

On the **frontend (visual)** → **No difference** (both bold).
On the **backend (semantic, SEO, accessibility)** → is meaningful, <b> is not.

2. Italic Text

This is italic

This is emphasized (semantic italic)

- → italic styling.
- → emphasizes text (important for accessibility).

3. Underline Text

<u>This is underlined</u>

4. Strikethrough (Deleted)

<strike>This is strikethrough (old)</strike>

~~This is deleted text~~

5. Inserted Text

<ins>This is inserted text</ins>

6. Superscript & Subscript

H²O <!-- Superscript -->

CO₂ <!-- Subscript -->

7. Monospace / Code

<tt>This is teletype (old)</tt>

<code>This is code text</code>

<pre>

Preformatted text

  (keeps spaces and line breaks)

</pre>

8. Highlighting

<mark>This is highlighted</mark>

Example with multiple formatting tags:

This is bold, italic, <u>underlined</u>,

<mark>highlighted</mark>, and H₂O with subscript.

Note:
Some tags like , , <u>, <tt>, <strike> are considered **presentational (old style)**. Modern HTML recommends **semantic tags** (, , <del>, <code>) because they carry meaning for browsers and screen readers.

Definition of List in HTML

A **list in HTML** is a structured way to group a collection of related items together. HTML provides different list types to display items **sequentially (numbered)**, **non-sequentially (bulleted)**, or with **terms and their descriptions**.

Key Points

- A list is made up of **list items (<li>)** or **terms (<dt>) and descriptions (<dd>)**.
- Lists make web content more **organized, readable, and accessible**.
- Three types of lists:
    1. **Ordered List (<ol>)** – items with numbers.
    2. **Unordered List (<ul>)** – items with bullets.
    3. **Description List (<dl>)** – terms with their descriptions.

Example definition (for students / exams):

"In HTML, a list is used to represent multiple related items in a structured format. It can be ordered (numbered), unordered (bulleted), or descriptive (term-definition)."

Ordered List

(ol, default is 1, style, nested, control List counting)

<html>

<head>

<title>

List in HTML

</title>

</head>

<body>

<h1> Ordered List </h1>

<ol type= "a" start ="5"> <!-- Control list counting-->

 Apple

```
<li> Mango </li>
<li> Banana </li>
<li> Guava </li>
</ol>

<!-- type = 1(default), type = "a", type = "A", type = "i", type = "I" (Uppercase Roman Number)-->
<h1> Nested Ordered List </h1>
<ol>
  <li>Coffee</li>
  <li>Tea
    <ol>
      <li>Black tea</li>
      <li>Green tea</li>
    </ol>
  </li>
  <li>Milk</li>
</ol>
</body>
</html>
```

Unordered List

(ul, default is bullet with filled circle, style, nested)

```
<html>
<head>
<title>
List in HTML
</title>
</head>
<body>
<h1> UnOrdered List </h1>
<ul type="disc">
<li> Apple </li>
<li> Mango </li>
<li> Banana </li>
<li> Guava </li>
</ul>

<ul type="circle">
<li> Apple </li>
<li> Mango </li>
<li> Banana </li>
<li> Guava </li>
</ul>

<ul type="square">
```

```
<li> Apple </li>
<li> Mango </li>
<li> Banana </li>
<li> Guava </li>
</ul>

<ul type="none">
<li> Apple </li>
<li> Mango </li>
<li> Banana </li>
<li> Guava </li>
</ul>
<h1> Nested Unordered List </h1>
<ul>
  <li>Coffee</li>
  <li>Tea
    <ul>
      <li>Black tea</li>
      <li>Green tea</li>
    </ul>
  </li>
  <li>Milk</li>
</ul>
</body>
</html>
```

Definition List

(dl, dt, dd)

```
<html>
<head>
<title>
List in HTML
</title>
</head>
<body>
<h1> Definition List </h1>
<dl>
<dt> Apple </dt>
<dd> Apple is a fruit. <br> An Apple a day keeps doctor away </dd>
<dt> Mango </dt>
<dd> Mango is a fruit. <br> It comes in many varieties. </dd>
</dl>
</body>
</html>
```

Image in HTML

The <img> tag is used to embed an image in an HTML page.

Tag <img>. It is an empty tag.

Images are not technically inserted into a web page; images are linked to web pages. The <img> tag creates a holding space for the referenced image.

The <img> tag has two required attributes:

- src - Specifies the path to the image
- alt - Specifies an alternate text for the image, if the image for some reason cannot be displayed

Note: Also, always specify the width and height of an image. If width and height are not specified, the page might flicker while the image loads.

src → path of the image file

1. **Import image that is saved on your computer**
   <img src="Path of the image">
   How to get the path of the image
   i. Go to the folder where the image is saved on your computer
   ii. For Example, it is saved on the desktop, then go to desktop
   iii. Then right click on the image, go to properties and then location
   iv. Copy the location path C:\Users\hp-d\Pictures
   v. Put a slash check the name of the file from the top written before type of file and type of file and write it after slash.
   vi. C:\Users\hp-d\Pictures\abc.jpg
2. **Import image from the internet**
   <img src="Link of the image">
   Go to google search for the image
   Right click on the image, copy image address.

   alt → text shown if the image can't load

   Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Simple Image Example</title>
```

```
</head>
<body>
 <h1>Image from Local Computer</h1>
 <img border = “2” src="E:\MITS\WebTechnology\abc.jfif" width="300" height="300">
 <h1>Image from Internet</h1>
 <img src="https://www.mitsgwalior.in/logo.jpg" alt="MITS LOGO" width="300" height="300">
</body>
</html>
```

height and width attributes

By default, the height and width attributes in the <img> tag are measured in **pixels (px)**.

Example:

```
<img src="photo.jpg" alt="Sample" width="300" height="200">
```

- width="300" → 300 pixels wide
- height="200" → 200 pixels tall

Important Notes:

1. **Default unit = pixels** (if you don’t specify any unit).
2. If you want **percentages (%)** or other CSS units (em, rem, vh, vw), you should use **CSS instead of attributes**:

```
<img src="photo.jpg" alt="Sample" style="width: 50%; height: auto;">
```

- Here 50% means half the width of the parent container.
- height: auto; keeps the image’s aspect ratio.

Q1. Write HTML code to display an image stored in your local computer inside the folder C:\Users\Pictures\flower.jpg with a width of 200px and height of 200px and border 4.

Q2. Write code to display an image from the internet. If the image is not available, the text "Image not found" should appear instead.

Q3. Write HTML code to display an image that links to https://www.wikipedia.org when clicked. The image should have a border of 3px.

Anchor <a> Tag in HTML

The <a> tag defines a hyperlink, which is used to link from one page to another.

The most important attribute of the <a> element is the href attribute, which indicates the link's destination.

By default, links will appear as follows in all browsers:

- An unvisited link is underlined and blue
- A visited link is underlined and purple
- An active link is underlined and red

If the <a> tag has no href attribute, it is only a placeholder for a hyperlink.

A linked page is normally displayed in the current browser window (parent window), unless you specify another target.

Example:

<html>

<head>

<title>

Anchor Tag

</title>

</head>

<body>

<h1> Target Blank </h1>

 Go to attribute

<h2> Target Parent and already saved file is taken </h1>

 Go to Image tag html file created earlier

<h2> a tag without href attribute </h2>

<a> No href attribute

```
</body>
</html>
```

<IMG> TAG INSIDE <A> TAG

```
<html >
<head>
  <title>Image Link Example</title>
</head>
<body>
  <h2>Click the Image to Open a Website</h2>
  <a href="https://www.mitsgwalior.in " target="_blank">
    <img src="https:// www.mitsgwalior.in /logo.jpg"
        alt="MITS LOGO"
        width="300" height="300" >
  </a>
</body>
</html>
```

Write an HTML program to create a web page that contains at least four hyperlinks using the <a> tag. One link should open in the same tab, one in a new tab, one linking to a local file, and one linking to a specific section of the same page.

HTML Tables

- The HTML tables allow web authors to arrange data like text, images, links, other tables, etc. into rows and columns of cells.
- The HTML tables are created using the **<table>** tag in which the **<tr>** tag is used to create table rows and **<td>** tag is used to create data cells.

Example

```
<html>
<head>
<title>HTML Tables</title>
</head>
<body>
<table border="1">
<tr>
<td>Row 1, Column 1</td>
<td>Row 1, Column 2</td>
</tr>
<tr>
<td>Row 2, Column 1</td>
<td>Row 2, Column 2</td>
</tr>
</table>
</body>
</html>
```

Table Heading

Table heading can be defined using **<th> tag.** This tag will be put to replace **<td> tag,** which is used to represent actual data cell. Normally you will put your top row as table heading as shown below, otherwise you can use <th> element in any row.

Example:

```
<html>
<head>
<title>HTML Table Header</title>
</head>
<body>
<table border="1">
<tr>
<th>Name</th>
<th>Salary</th>
</tr>
<tr>
<td>XYZ</td>
<td>5000</td>
</tr>
<tr>
<td>ABC</td>
<td>7000</td>
</tr>
</table>
```

```
</body>

</html>
```

Table Height and Width

You can set a table width and height using **width** and **height** attributes. You can specify table width or height in terms of pixels or in terms of percentage of available screen area.

```
<html>

<head>

<title>HTML Table Width/Height</title>

</head>

<body>

<table border="1" width="400" height="150">

<tr>

<td>Row 1, Column 1</td>

<td>Row 1, Column 2</td>

</tr>

<tr>

<td>Row 2, Column 1</td>

<td>Row 2, Column 2</td>

</tr>

</table>

</body>

</html>
```

HTML Form

The <form> tag is used to create an **HTML form**. Forms are used to collect input from the user (e.g., login, registration).

<input> is an HTML tag (also called an element).

- It is a self-closing tag, meaning it doesn't need a separate closing tag like ... .
- It is used inside forms to accept user input (text, password, checkbox, radio button, file upload, etc.).

Name: Each form input needs a unique name so the server knows which value belongs to which field.

```
<html>
<head>
<title>
form
</title>
</head>
<body>
<form>
<input type="text" name="username" placeholder="Enter enrollment">
<input type="password" name="pass" placeholder="Enter password">
<input type="submit" name="login">
</form>
</body>
</html>
```

Example:

```
<html>
<head>
<title>
form
</title>
</head>
<body>
<form>
<h1> LOGIN PAGE </h1>
USERNAME <input type="text" name="username" placeholder="Enter enrollment">
<br><br><br><br>
PASSWORD <input type="password" name="pass" placeholder="Enter password">
<br><br><br><br>
<input type="submit" name="login">
</form>
</body>
</html>
```

Part	**Meaning**
<input>	**Tag** used to create an input field where the user can enter data. It is an **empty/self-closing-type element**, so it does not need </input>.
type="text"	**Attribute** that specifies the type of input. text creates a single-line text box.
name="username"	**Attribute** that gives the input field a name/identifier. It is especially important when the form data is submitted to a server.
placeholder="Enter enrollment"	**Attribute** that displays temporary hint text inside the input box. It disappears when the user starts typing.

type value	Purpose	Example
text	Single-line text	<input type="text">
password	Password input; characters are hidden	<input type="password">
email	Email address	<input type="email">
number	Numeric input	<input type="number">
tel	Telephone/mobile number	<input type="tel">
url	Website URL	<input type="url">
date	Select a date	<input type="date">
time	Select a time	<input type="time">
datetime-local	Date and time	<input type="datetime-local">
month	Select month and year	<input type="month">
week	Select a week	<input type="week">
radio	Select **one** option from a group	<input type="radio">
checkbox	Select **one or more** options	<input type="checkbox">
file	Select/upload a file	<input type="file">
color	Select a color	<input type="color">
range	Select a value using a slider	<input type="range">
search	Search field	<input type="search">
hidden	Hidden input field	<input type="hidden">
submit	Submit a form	<input type="submit">
reset	Reset form fields	<input type="reset">
button	Ordinary clickable button	<input type="button">
image	Image used as a submit button	<input type="image">

Dr. Shradha Dubey
Assistant Professor
Department of CCST

<textarea> Tag in HTML Form

The **<textarea>** tag is used when the user needs to enter **multiple lines of text**.

For example: address, feedback, comments, description, etc.

Basic Syntax

```
<textarea></textarea>
```

Example: Address

```
<form>
  <label for="address">Address:</label><br>
  <textarea id="address" name="address" rows="5" cols="40"></textarea>
</form>
```

Important Attributes

Attribute	Purpose
id	Uniquely identifies the textarea
name	Gives a name to the form field
rows	Specifies the visible height
cols	Specifies the visible width
placeholder	Displays a hint inside the textarea

Example with Placeholder

```
<label for="feedback">Feedback:</label><br>
<textarea
  id="feedback"
  name="feedback"
  rows="5"
  cols="40"
  placeholder="Enter your feedback here">
```

</textarea>

<textarea>Text goes here</textarea>

For example, the default text can be written as:

<textarea rows="4" cols="30">Enter your message...</textarea>

Here, **rows controls the height** and **cols controls the width** of the textarea.

Radio Button in HTML Form

A radio button is used when the user needs to select only one option from a group of choices.

It is created using:

<input type="radio">

Example: Gender Selection

```
<form>
  <label>Gender:</label><br>

  <input type="radio" id="male" name="gender" value="Male">
  <label for="male">Male</label>

  <input type="radio" id="female" name="gender" value="Female">
  <label for="female">Female</label>

  <input type="radio" id="other" name="gender" value="Other">
  <label for="other">Other</label>
</form>
```

Important Attributes

Attribute	**Purpose**
type="radio"	Creates a radio button
name="gender"	Groups the radio buttons
value="Male"	Value associated with the selected option
id="male"	Uniquely identifies the radio button
for="male"	Connects the label with the radio button

Note: All radio buttons that belong to the same group must have the **same name**.

For example:

name="gender"

for Male, Female, and Other allows the user to select **only one**.

Checkbox in HTML Form

A **checkbox** is used when the user can select **one or more options** from a list.

It is created using:

<input type="checkbox">

Example: Select Hobbies

```
<form>
  <label>Hobbies:</label><br>

  <input type="checkbox" id="reading" name="hobby" value="Reading">
  <label for="reading">Reading</label><br>

  <input type="checkbox" id="music" name="hobby" value="Music">
  <label for="music">Music</label><br>

  <input type="checkbox" id="sports" name="hobby" value="Sports">
  <label for="sports">Sports</label><br>

  <input type="checkbox" id="coding" name="hobby" value="Coding">
  <label for="coding">Coding</label>
</form>
```

Important Attributes

Attribute	**Purpose**
type="checkbox"	Creates a checkbox
id	Uniquely identifies the checkbox
name	Gives a name to the form field
value	Specifies the value of the selected option

Attribute	Purpose
checked	Makes the checkbox selected by default

Checkbox vs Radio Button

- **Checkbox:** Multiple options can be selected
- **Radio button:** Only one option can be selected

Label Tag

<label> Tag in HTML

The <label> tag is used to provide a **text description/name for an input field**.

For example:

```
<label>Enrollment Number:</label>
<input type="text" name="enrollment">
```

Here:

- <label> → tells the user what information to enter.
- Enrollment Number: → the text displayed to the user.
- <input> → creates the box where the user enters the enrollment number.

Why is <label> important?

1. It tells the user what to enter

Without a label:

```
<input type="text">
```

The user sees only a blank box.

With a label:

```
<label>Enrollment Number:</label>
<input type="text">
```

It is clear what the field is for.

2. It improves accessibility

Screen readers can use labels to tell users what an input field represents. This is especially important for users with visual disabilities.

3. It can make the input easier to select

The recommended way is to connect the label with the input using for and id:

```
<label for="enroll">Enrollment Number:</label>
<input type="text" id="enroll" name="username">
```

Here:

- for="enroll" → identifies which input the label belongs to.
- id="enroll" → uniquely identifies the input.

The values of for and id should **match**.

So:

for="enroll" → id="enroll"

When the user clicks **"Enrollment Number:"**, the cursor is placed in the corresponding input box.

<label> tells the user WHAT to enter, while <input> provides the place WHERE to enter it.

Label = Description
Input = Data entry field

Attribute	Main purpose	Used by
id	Uniquely identifies the HTML element	HTML, CSS, JavaScript, <label>
name	Identifies the field/data during form submission	Form/server

Dropdown in HTML

A **dropdown list** allows the user to select **one option from a list of choices**.

It is created using the <select> and <option> tags.

Basic Syntax

```
<label for="course">Select Course:</label>

<select id="course" name="course">
   <option value="cse">Computer Science</option>
   <option value="it">Information Technology</option>
   <option value="ece">Electronics</option>
   <option value="me">Mechanical</option>
</select>
```

How it works

- <select> → creates the dropdown box.
- <option> → creates each choice inside the dropdown.
- value → the value associated with the selected option.
- id → connects the dropdown with the <label>.
- name → identifies the field when form data is submitted.

Dropdown in a Student Registration Form

```
<form>

   <label for="name">Student Name:</label>
   <input type="text" id="name" name="name">
   <br><br>

   <label for="branch">Select Branch:</label>
   <select id="branch" name="branch">
      <option value="">--Select Branch--</option>
      <option value="cse">CSE</option>
      <option value="ece">ECE</option>
      <option value="me">Mechanical</option>
      <option value="ce">Civil</option>
   </select>

   <br><br>

   <input type="submit" value="Register">
```

</form>

Important: value vs displayed text

<option value="cse">Computer Science Engineering</option>

Here:

- **Displayed to user:** Computer Science Engineering
- **Actual value:** cse

Practice Question

Create a Student Registration Form containing:

1. Student Name
2. Enrollment Number
3. Email
4. Gender — Radio buttons
5. Branch — Dropdown with CSE, ECE, ME, CE
6. Semester — Dropdown from 1 to 8
7. Skills — Checkboxes
8. Address — Textarea
9. Submit and Reset buttons

File/ Photo Upload

To **upload a photo in an HTML form**, use the <input> tag with type="file".

Basic Example

```
<form>
   <label for="photo">Upload Photo:</label>
   <input type="file" id="photo" name="photo">

   <br><br>

   <input type="submit" value="Submit">
</form>
```

Important attributes

- type="file" → creates a file-upload button.
- id="photo" → connects the input with the <label>.
- name="photo" → identifies the uploaded file when the form is submitted.

Only allow image files

```
<form>
   <label for="photo">Upload Profile Photo:</label>
   <input type="file" id="photo" name="photo" accept="image/*">

   <br><br>

   <input type="submit" value="Register">
</form>
```

Here, accept="image/*" means the user can select **image files only**.

Student Registration Form with Photo

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Registration</title>
</head>
<body>

<h2>Student Registration Form</h2>

<form>

   <label for="name">Student Name:</label>
   <input type="text" id="name" name="name">

   <br><br>

   <label for="enrollment">Enrollment Number:</label>
   <input type="text" id="enrollment" name="enrollment">

   <br><br>

   <label for="email">Email:</label>
   <input type="email" id="email" name="email">

   <br><br>

   <label for="photo">Upload Photo:</label>
   <input type="file" id="photo" name="photo" accept="image/*">

   <br><br>

   <input type="submit" value="Register">
   <input type="reset" value="Reset">

</form>

</body>
</html>
```

Practice question: Create a **Student Admission Form** containing Name, Enrollment Number, Email, Mobile Number, Gender, Date of Birth, Course (dropdown), Address (textarea), **Profile Photo (upload)**, Submit, and Reset buttons.

accept value	Allows
image/*	All images
video/*	All videos

accept value	Allows
audio/*	All audio files
.pdf	PDF
.doc, .docx	Word documents
.xls, .xlsx	Excel files
.txt	Text files
.jpg, .png	Specific image formats